

☒ Mini D-Mart — Enterprise-Grade Full-Stack Grocery Platform

Complete Development Specification

Version: 1.0.0

Last Updated: August 21, 2026

Status: Production-Ready Specification

Table of Contents

1. [Executive Summary](#)
2. [Technology Stack](#)
3. [User Roles & Permissions](#)
4. [Project Architecture](#)
5. [Database Schema](#)
6. [API Specification](#)
7. [Authentication & Authorization](#)
8. [Business Logic](#)
9. [Security Requirements](#)
10. [Testing Strategy](#)
11. [Deployment Configuration](#)
12. [Performance Requirements](#)
13. [Monitoring & Observability](#)
14. [Development Phases](#)
15. [Frontend Routes](#)
16. [UI/UX Requirements](#)
17. [Known Limitations](#)
18. [Getting Started](#)
19. [Test Credentials](#)
20. [Success Criteria](#)
21. [Final Deliverables](#)

1. Executive Summary

Build a production-ready, scalable grocery e-commerce platform with three distinct user roles (Customer, Staff, Admin), complete order lifecycle management, returns/exchanges processing, and comprehensive audit logging. The system must handle concurrent users, prevent overselling, and maintain data integrity through proper transaction management.

Core Features

- **Multi-Role System:** Customer, Staff, and Admin with distinct permissions
- **Product Management:** Categories, products, search, filters, inventory tracking
- **Shopping Cart:** Add, update, remove items with real-time stock validation
- **Checkout System:** Atomic transactions with stock reservation
- **Order Management:** Pickup and delivery fulfillment with status workflows
- **Returns & Exchanges:** Eligibility validation and approval workflows
- **Dashboards:** Role-specific dashboards with analytics
- **Audit Logging:** Comprehensive tracking of all important actions

Key Technical Requirements

- **Concurrency:** Handle 100+ simultaneous checkouts without overselling
- **Security:** JWT authentication, RBAC, rate limiting, input validation
- **Performance:** API responses < 200ms (95th percentile), pages < 3s load time
- **Reliability:** 99.9% uptime, proper error handling, graceful degradation
- **Scalability:** Containerized deployment, horizontal scaling ready

2. Technology Stack

2.1 Frontend

Technology	Version	Purpose
React	18+	UI Framework
TypeScript	5+	Type Safety
Vite	5+	Build Tool
React Router DOM	6+	Routing
Axios	1+	HTTP Client
Tailwind CSS	3+	Styling
Headless UI	Latest	UI Primitives
React Hook Form	7+	Form Handling

Technology	Version	Purpose
Zod	3+	Validation
React Hot Toast	Latest	Notifications
Lucide React	Latest	Icons
date-fns	2+	Date Utilities
Vitest	Latest	Unit Testing
React Testing Library	Latest	Component Testing
Playwright	Latest	E2E Testing

2.2 Backend

Technology	Version	Purpose
Python	3.11+	Language
Django	5+	Web Framework
Django REST Framework	3.14+	API Framework
djangorestframework-simplejwt	Latest	JWT Auth
django-cors-headers	Latest	CORS
django-ratelimit	Latest	Rate Limiting
django-filter	Latest	API Filtering
drf-spectacular	Latest	OpenAPI Docs
pytest	7+	Testing
pytest-django	Latest	Django Testing
pytest-cov	Latest	Coverage
black	Latest	Code Formatting
isort	Latest	Import Sorting
flake8	Latest	Linting
mypy	Latest	Type Checking

2.3 Database & Caching

Technology	Version	Purpose
PostgreSQL	15+	Primary Database
Redis	7+	Caching, Sessions, Rate Limiting
psycopg2-binary	Latest	PostgreSQL Adapter
pgBouncer	Latest	Connection Pooling (Production)

2.4 DevOps & Infrastructure

Technology	Purpose
Docker	Containerization
Docker Compose	Local Development
GitHub Actions	CI/CD
Vercel	Frontend Hosting
Render/Railway	Backend Hosting
Sentry	Error Tracking
Grafana/Prometheus	Monitoring (Optional)
ELK Stack	Log Aggregation (Optional)

3. User Roles & Permissions

3.1 Role Definitions

Role	Description	Access Level
Customer	End users who browse and purchase products	Limited to own data
Staff	Store employees who process orders and returns	Operational access
Admin	System administrators with full control	Full system access

3.2 Permissions Matrix

Feature	Customer	Staff	Admin
Browse products	✓	✓	✓
Search products	✓	✓	✓
Filter products	✓	✓	✓
View product details	✓	✓	✓
Manage own cart	✓	✗	✗
Place orders	✓	✗	✗
View own orders	✓	✗	✗
Cancel own orders (eligible)	✓	✗	✗
Request returns	✓	✗	✗
Request exchanges	✓	✗	✗
View own return requests	✓	✓	✓

Feature	Customer	Staff	Admin
View all orders	✗	✓	✓
Update order status	✗	✓ (restricted)	✓ (all)
View upcoming pickups	✗	✓	✓
View delivery orders	✗	✓	✓
Process returns/exchanges	✗	✓ (review)	✓ (approve/reject)
View inventory	✗	✓	✓
Manage products	✗	✗	✓
Manage categories	✗	✗	✓
Manage inventory	✗	✗	✓
Manage users	✗	✗	✓
Assign roles	✗	✗	✓
Manage pickup slots	✗	✗	✓
View audit logs	✗	✗	✓
View low-stock alerts	✗	✓	✓

3.3 Resource Ownership Rules

- **Customers** can only access their own:
 - Cart
 - Orders
 - Return/exchange requests
 - Profile data
- **Staff** can access:
 - All orders (read-only except status updates)
 - All return/exchange requests (process)
 - Inventory data (read-only)
- **Admins** can access:
 - All system resources
 - Audit logs
 - User management
 - System configuration

4. Project Architecture

4.1 Directory Structure

```
mini-dmart/
├── frontend/
│   ├── public/
│   │   ├── favicon.ico
│   │   └── manifest.json
│   └── src/
│       ├── api/
│       │   ├── client.ts
│       │   ├── auth.ts
│       │   ├── products.ts
│       │   ├── cart.ts
│       │   ├── orders.ts
│       │   └── returns.ts
│       ├── components/
│       │   ├── ui/
│       │   │   ├── Button.tsx
│       │   │   ├── Input.tsx
│       │   │   ├── Modal.tsx
│       │   │   ├── Toast.tsx
│       │   │   └── Loader.tsx
│       │   ├── layout/
│       │   │   ├── Navbar.tsx
│       │   │   ├── Sidebar.tsx
│       │   │   ├── Footer.tsx
│       │   │   └── ProtectedRoute.tsx
│       │   ├── products/
│       │   │   ├── ProductCard.tsx
│       │   │   ├── ProductGrid.tsx
│       │   │   ├── ProductFilters.tsx
│       │   │   └── ProductSearch.tsx
│       │   ├── cart/
│       │   │   ├── CartItem.tsx
│       │   │   ├── CartSummary.tsx
│       │   │   └── EmptyCart.tsx
│       │   ├── orders/
│       │   │   ├── OrderCard.tsx
│       │   │   ├── OrderStatus.tsx
│       │   │   └── OrderTimeline.tsx
│       │   └── returns/
│       │       ├── ReturnRequestForm.tsx
│       │       └── ReturnStatus.tsx
│       ├── context/
│       │   ├── AuthContext.tsx
│       │   ├── CartContext.tsx
│       │   └── ThemeContext.tsx
│       └── hooks/
│           ├── useAuth.ts
│           ├── useCart.ts
│           ├── useDebounce.ts
│           └── useLocalStorage.ts
```

```
├── layouts/
│   ├── AuthLayout.tsx
│   ├── CustomerLayout.tsx
│   ├── StaffLayout.tsx
│   └── AdminLayout.tsx
├── pages/
│   ├── auth/
│   │   ├── LoginPage.tsx
│   │   ├── RegisterPage.tsx
│   │   └── ForgotPasswordPage.tsx
│   ├── customer/
│   │   ├── HomePage.tsx
│   │   ├── ProductsPage.tsx
│   │   ├── ProductDetailPage.tsx
│   │   ├── CartPage.tsx
│   │   ├── CheckoutPage.tsx
│   │   ├── OrdersPage.tsx
│   │   ├── OrderDetailPage.tsx
│   │   ├── ReturnsPage.tsx
│   │   └── ProfilePage.tsx
│   ├── staff/
│   │   ├── DashboardPage.tsx
│   │   ├── OrdersPage.tsx
│   │   ├── PickupsPage.tsx
│   │   ├── DeliveriesPage.tsx
│   │   └── ReturnsPage.tsx
│   └── admin/
│       ├── DashboardPage.tsx
│       ├── ProductsPage.tsx
│       ├── CategoriesPage.tsx
│       ├── UsersPage.tsx
│       ├── OrdersPage.tsx
│       ├── InventoryPage.tsx
│       ├── AuditLogsPage.tsx
│       └── PickupSlotsPage.tsx
├── routes/
│   ├── index.tsx
│   ├── customerRoutes.tsx
│   ├── staffRoutes.tsx
│   └── adminRoutes.tsx
├── types/
│   ├── auth.ts
│   ├── product.ts
│   ├── order.ts
│   └── api.ts
├── utils/
│   ├── formatters.ts
│   ├── validators.ts
│   └── constants.ts
├── App.tsx
├── main.tsx
├── index.css
├── .env.example
├── .env.local
├── index.html
└── package.json
```

```
├─ tsconfig.json
├─ tailwind.config.js
├─ vite.config.ts
└─ README.md
```

```

└─ backend/
    └─ config/
        ├── __init__.py
        ├── asgi.py
        ├── settings/
        │   ├── __init__.py
        │   ├── base.py
        │   ├── development.py
        │   └── production.py
        ├── urls.py
        └── wsgi.py
    └─ apps/
        ├── __init__.py
        ├── accounts/
        │   ├── __init__.py
        │   ├── models.py
        │   ├── serializers.py
        │   ├── views.py
        │   ├── urls.py
        │   ├── permissions.py
        │   ├── authentication.py
        │   └── tests/
        ├── products/
        │   ├── __init__.py
        │   ├── models.py
        │   ├── serializers.py
        │   ├── views.py
        │   ├── urls.py
        │   ├── filters.py
        │   └── tests/
        ├── cart/
        │   ├── __init__.py
        │   ├── models.py
        │   ├── serializers.py
        │   ├── views.py
        │   ├── urls.py
        │   └── tests/
        ├── orders/
        │   ├── __init__.py
        │   ├── models.py
        │   ├── serializers.py
        │   ├── views.py
        │   ├── urls.py
        │   ├── services.py
        │   └── tests/
        └── returns_exchange/
            ├── __init__.py
            ├── models.py
            ├── serializers.py
            ├── views.py
            └── urls.py

```



```

├── tests/
├── operations/
│   ├── __init__.py
│   ├── models.py
│   ├── serializers.py
│   ├── views.py
│   └── urls.py
├── audit/
│   ├── __init__.py
│   ├── models.py
│   ├── serializers.py
│   ├── views.py
│   └── urls.py
├── manage.py
├── requirements/
│   ├── base.txt
│   ├── development.txt
│   └── production.txt
├── .env.example
├── .env.local
├── pytest.ini
├── README.md
├── docker/
│   ├── Dockerfile.frontend
│   ├── Dockerfile.backend
│   └── docker-compose.yml
├── .github/
│   └── workflows/
│       ├── ci.yml
│       └── deploy.yml
├── .gitignore
├── .editorconfig
├── .pre-commit-config.yaml
├── README.md
├── SECURITY.md
├── CHANGELOG.md
├── CONTRIBUTING.md
├── LICENSE
├── docs/
│   ├── api/
│   ├── architecture/
│   └── deployment/

```

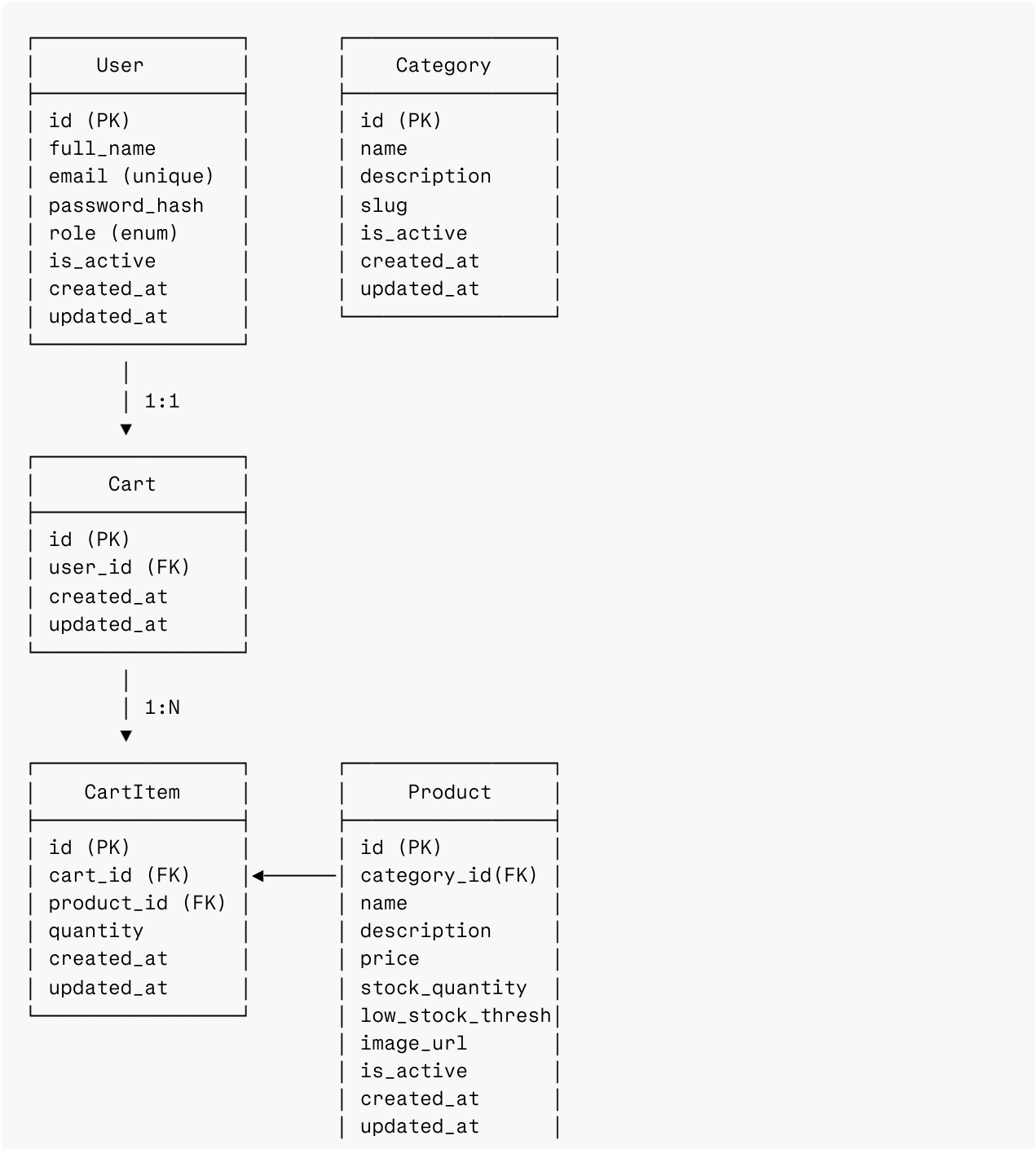
4.2 Backend App Separation

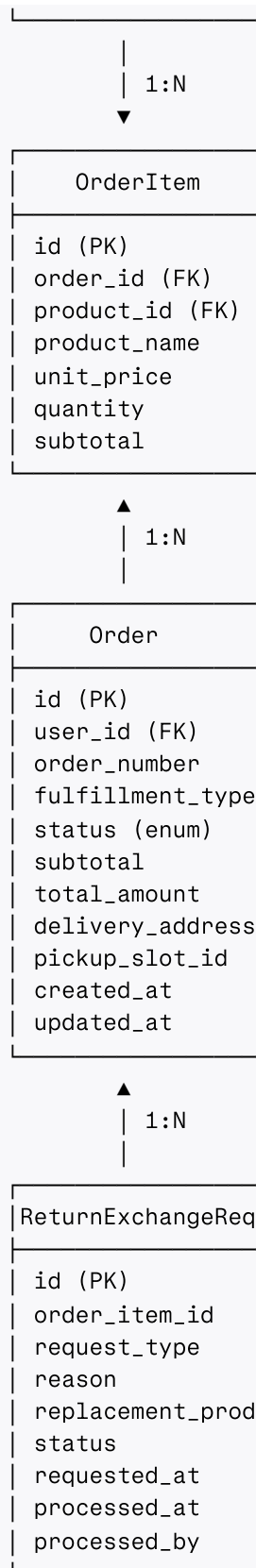
App	Responsibility
accounts	User authentication, registration, profile, roles
products	Categories, products, inventory, search, filters
cart	Shopping cart management, cart items

App	Responsibility
orders	Order creation, checkout, status management
returns_exchange	Return and exchange request processing
operations	Pickup slots, staff operations
audit	Audit logging, admin logs

5. Database Schema

5.1 Entity Relationship Diagram





5.2 Model Specifications

User

```
id: UUID (Primary Key)
full_name: String (max 255)
email: String (unique, max 255)
password_hash: String
role: Enum (CUSTOMER, STAFF, ADMIN)
is_active: Boolean (default: True)
created_at: DateTime
updated_at: DateTime
```

Category

```
id: UUID (Primary Key)
name: String (max 100)
description: Text (optional)
slug: String (unique, auto-generated)
is_active: Boolean (default: True)
created_at: DateTime
updated_at: DateTime
```

Product

```
id: UUID (Primary Key)
category: ForeignKey(Category)
name: String (max 255)
description: Text
price: Decimal (>= 0)
stock_quantity: Integer (>= 0)
low_stock_threshold: Integer (>= 0, default: 10)
image_url: String (URL)
is_active: Boolean (default: True)
created_at: DateTime
updated_at: DateTime
```

Cart

```
id: UUID (Primary Key)
user: OneToOne(User)
created_at: DateTime
updated_at: DateTime
```

CartItem

```
id: UUID (Primary Key)
cart: ForeignKey(Cart)
product: ForeignKey(Product)
quantity: Integer (> 0)
created_at: DateTime
updated_at: DateTime

# Unique Constraint: (cart, product)
```

PickupSlot

```
id: UUID (Primary Key)
date: Date
start_time: Time
end_time: Time
capacity: Integer (> 0)
is_active: Boolean (default: True)
created_at: DateTime
```

Order

```
id: UUID (Primary Key)
user: ForeignKey(User)
order_number: String (unique, auto-generated)
fulfillment_type: Enum (PICKUP, DELIVERY)
status: Enum (PENDING, CONFIRMED, PREPARING, READY_FOR_PICKUP,
              OUT_FOR_DELIVERY, COMPLETED, DELIVERED, CANCELLED)
subtotal: Decimal
total_amount: Decimal
delivery_address: JSON (nullable)
pickup_slot: ForeignKey(PickupSlot, nullable)
created_at: DateTime
updated_at: DateTime
```

OrderItem

```
id: UUID (Primary Key)
order: ForeignKey(Order)
product: ForeignKey(Product)
product_name: String (snapshot)
unit_price: Decimal (snapshot)
quantity: Integer
subtotal: Decimal
```

ReturnExchangeRequest

```
id: UUID (Primary Key)
order_item: ForeignKey(OrderItem)
request_type: Enum (RETURN, EXCHANGE)
reason: Text
replacement_product: ForeignKey(Product, nullable)
status: Enum (REQUESTED, APPROVED, REJECTED, COMPLETED)
requested_at: DateTime
processed_at: DateTime (nullable)
processed_by: ForeignKey(User, nullable)
```

AuditLog

```
id: UUID (Primary Key)
user: ForeignKey(User)
action: String
entity_type: String
entity_id: UUID
metadata: JSON
created_at: DateTime
```

5.3 Database Indexes

```
-- Performance indexes
CREATE INDEX idx_products_category ON products(category_id) WHERE is_active = true;
CREATE INDEX idx_products_price ON products(price);
CREATE INDEX idx_products_stock ON products(stock_quantity);
CREATE INDEX idx_orders_user ON orders(user_id);
CREATE INDEX idx_orders_status ON orders(status);
CREATE INDEX idx_orders_created ON orders(created_at DESC);
CREATE INDEX idx_orders_fulfillment ON orders(fulfillment_type);
CREATE INDEX idx_cart_items_cart ON cart_items(cart_id);
CREATE INDEX idx_cart_items_product ON cart_items(product_id);
CREATE INDEX idx_audit_logs_user ON audit_logs(user_id);
CREATE INDEX idx_audit_logs_entity ON audit_logs(entity_type, entity_id);
CREATE INDEX idx_pickup_slots_date ON pickup_slots(date);
CREATE INDEX idx_returns_status ON return_exchange_request(status);

-- Unique constraints
ALTER TABLE cart_items ADD CONSTRAINT unique_cart_product UNIQUE (cart_id, product_id);
ALTER TABLE users ADD CONSTRAINT unique_email UNIQUE (email);
ALTER TABLE orders ADD CONSTRAINT unique_order_number UNIQUE (order_number);
```

6. API Specification

6.1 Base Configuration

- **Base URL:** /api/v1/
- **Authentication:** JWT Bearer Token
- **Content-Type:** application/json
- **Response Format:** Standardized JSON

6.2 Standard Response Format

Success Response:

```
{
  "success": true,
  "message": "Operation completed successfully",
  "data": {}
}
```

Error Response:

```
{
  "success": false,
  "message": "Error description",
  "errors": {
    "field_name": ["Error message"]
  }
}
```

6.3 HTTP Status Codes

Code	Meaning	Usage
200	OK	Successful GET, PUT, PATCH
201	Created	Successful POST (resource creation)
400	Bad Request	Validation errors
401	Unauthorized	Missing or invalid token
403	Forbidden	Insufficient permissions
404	Not Found	Resource not found
409	Conflict	Business logic conflicts (stock, capacity)
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Server errors

6.4 Authentication Endpoints

POST /auth/register/

Register a new user account.

Request:

```
{
  "full_name": "John Doe",
  "email": "john@example.com",
  "password": "SecureP@ssw0rd123",
  "confirm_password": "SecureP@ssw0rd123"
}
```

Response (201):

```
{
  "success": true,
  "message": "Registration successful",
  "data": {
    "user": {
      "id": "uuid",
      "full_name": "John Doe",
      "email": "john@example.com",
      "role": "CUSTOMER"
    },
    "tokens": {
      "access": "eyJ0eXAiOiJKV1QiLCJhbG...",
      "refresh": "eyJ0eXAiOiJKV1QiLCJhbG..."
    }
  }
}
```

Validation Rules:

- `full_name`: Required, 2-255 characters
- `email`: Required, valid email format, unique
- `password`: Required, min 8 chars, 1 uppercase, 1 lowercase, 1 digit, 1 special char
- `confirm_password`: Required, must match password

POST /auth/login/

Authenticate user and obtain tokens.

Request:

```
{
  "email": "john@example.com",
}
```



```
"password": "SecureP@ssw0rd123"
}
```

Response (200):

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "user": {
      "id": "uuid",
      "full_name": "John Doe",
      "email": "john@example.com",
      "role": "CUSTOMER"
    },
    "tokens": {
      "access": "eyJ0eXAiOiJKV1QiLCJhbG...",
      "refresh": "eyJ0eXAiOiJKV1QiLCJhbG..."
    }
  }
}
```

Rate Limiting: 5 attempts per 15 minutes per IP

POST /auth/token/refresh/

Refresh access token.

Request:

```
{
  "refresh": "eyJ0eXAiOiJKV1QiLCJhbG..."
}
```

Response (200):

```
{
  "success": true,
  "message": "Token refreshed successfully",
  "data": {
    "access": "eyJ0eXAiOiJKV1QiLCJhbG..."
  }
}
```

GET /auth/profile/

Get current user profile.

Response (200):

```
{
  "success": true,
  "message": "Profile retrieved successfully",
  "data": {
    "id": "uuid",
    "full_name": "John Doe",
    "email": "john@example.com",
    "role": "CUSTOMER",
    "created_at": "2024-08-21T10:30:00Z"
  }
}
```

PUT /auth/profile/

Update current user profile.

Request:

```
{
  "full_name": "John Updated",
  "email": "john.updated@example.com"
}
```

6.5 Product Endpoints

GET /categories/

List all active categories.

Response (200):

```
{
  "success": true,
  "message": "Categories retrieved successfully",
  "data": [
    {
      "id": "uuid",
      "name": "Dairy",
      "description": "Dairy products",
      "slug": "dairy"
    }
  ]
}
```

GET /products/

List products with filtering and pagination.

Query Parameters:

- `search` (string): Search in name and description
- `category` (integer): Filter by category ID
- `min_price` (number): Minimum price
- `max_price` (number): Maximum price
- `in_stock` (boolean): Filter in-stock items
- `page` (integer): Page number (default: 1)
- `page_size` (integer): Items per page (default: 20, max: 100)

Response (200):

```
{
  "success": true,
  "message": "Products retrieved successfully",
  "data": {
    "products": [
      {
        "id": "uuid",
        "name": "Organic Milk",
        "description": "Fresh organic whole milk",
        "price": 4.99,
        "stock_quantity": 150,
        "low_stock_threshold": 20,
        "category": {
          "id": "uuid",
          "name": "Dairy",
          "slug": "dairy"
        },
        "image_url": "https://...",
        "is_in_stock": true
      }
    ],
    "pagination": {
      "page": 1,
      "page_size": 20,
      "total_pages": 5,
      "total_items": 98
    }
  }
}
```

GET /products/{id}/

Get product details.

Response (200):

```
{
  "success": true,
  "message": "Product retrieved successfully",
  "data": {
    "id": "uuid",
    "name": "Organic Milk",
    "description": "Fresh organic whole milk",
    "price": 4.99,
    "stock_quantity": 150,
    "category": {
      "id": "uuid",
      "name": "Dairy",
      "slug": "dairy"
    },
    "image_url": "https://...",
    "is_in_stock": true
  }
}
```

POST /products/ (Admin Only)

Create a new product.

Request:

```
{
  "category_id": "uuid",
  "name": "Organic Milk",
  "description": "Fresh organic whole milk",
  "price": 4.99,
  "stock_quantity": 100,
  "low_stock_threshold": 20,
  "image_url": "https://..."
}
```

PUT /products/{id}/ (Admin Only)

Update product.

DELETE /products/{id}/ (Admin Only)

Deactivate product (soft delete).

6.6 Cart Endpoints

GET /cart/

Get current user's cart.

Response (200):

```
{
  "success": true,
  "message": "Cart retrieved successfully",
  "data": {
    "id": "uuid",
    "items": [
      {
        "id": "uuid",
        "product": {
          "id": "uuid",
          "name": "Organic Milk",
          "price": 4.99,
          "image_url": "https://..."
        },
        "quantity": 2,
        "subtotal": 9.98
      }
    ],
    "subtotal": 9.98,
    "total_items": 2
  }
}
```

POST /cart/items/

Add item to cart.

Request:

```
{
  "product_id": "uuid",
  "quantity": 2
}
```

Response (201):

```
{
  "success": true,
  "message": "Item added to cart successfully",
  "data": {
    "id": "uuid",
    "product": {...},
    "quantity": 2,
  }
}
```

```
    "subtotal": 9.98
  }
}
```

Error (409 - Insufficient Stock):

```
{
  "success": false,
  "message": "Insufficient stock available",
  "errors": {
    "quantity": ["Only 5 items available in stock"]
  }
}
```

PATCH /cart/items/{id}/

Update cart item quantity.

Request:

```
{
  "quantity": 3
}
```

DELETE /cart/items/{id}/

Remove item from cart.

DELETE /cart/clear/

Clear entire cart.

6.7 Checkout & Order Endpoints

POST /orders/checkout/

Process checkout and create order.

Request (Pickup):

```
{
  "fulfillment_type": "PICKUP",
  "pickup_slot_id": "uuid",
  "delivery_address": null
}
```

Request (Delivery):

```
{
  "fulfillment_type": "DELIVERY",
  "pickup_slot_id": null,
  "delivery_address": {
    "street": "123 Main St",
    "city": "Springfield",
    "state": "IL",
    "postal_code": "62701",
    "contact_number": "+1-555-123-4567"
  }
}
```

Response (201):

```
{
  "success": true,
  "message": "Order placed successfully",
  "data": {
    "order": {
      "id": "uuid",
      "order_number": "ORD-2024-000001",
      "fulfillment_type": "PICKUP",
      "status": "PENDING",
      "subtotal": 49.90,
      "total_amount": 49.90,
      "items_count": 5,
      "pickup_slot": {
        "date": "2024-08-25",
        "start_time": "14:00",
        "end_time": "16:00"
      },
      "created_at": "2024-08-21T10:30:00Z"
    }
  }
}
```

Checkout Process (Atomic Transaction):

1. Retrieve active cart with items
2. Validate cart is not empty
3. Validate all products exist and are active
4. Validate stock quantities
5. Validate pickup slot capacity (if applicable)
6. Calculate subtotal and total
7. Create order record
8. Create order items (snapshot product data)
9. Reduce product stock quantities
10. Clear cart items

11. Create audit log entry

12. Commit transaction

Error Scenarios:

- Empty cart → 400 Bad Request
- Insufficient stock → 409 Conflict
- Full pickup slot → 409 Conflict
- Invalid pickup slot → 404 Not Found

GET /orders/

List user's orders (Customer) or all orders (Staff/Admin).

Query Parameters:

- `status` (string): Filter by status
- `fulfillment_type` (string): PICKUP or DELIVERY
- `page` (integer): Page number
- `page_size` (integer): Items per page

GET /orders/{id}/

Get order details.

Authorization:

- Customers: Own orders only
- Staff/Admin: All orders

POST /orders/{id}/cancel/

Cancel order (Customer, eligible orders only).

Eligibility:

- Status: PENDING or CONFIRMED
- Not already cancelled/completed

Response (200):

```
{
  "success": true,
  "message": "Order cancelled successfully",
  "data": {
    "order": {
      "id": "uuid",
      "status": "CANCELLED"
    }
  }
}
```



```
}  
}
```

PATCH /orders/{id}/status/ (Staff/Admin Only)

Update order status.

Request:

```
{  
  "status": "CONFIRMED"  
}
```

Valid Transitions:

- PENDING → CONFIRMED, CANCELLED
- CONFIRMED → PREPARING, CANCELLED
- PREPARING → READY_FOR_PICKUP, OUT_FOR_DELIVERY
- READY_FOR_PICKUP → COMPLETED
- OUT_FOR_DELIVERY → DELIVERED

Error (Invalid Transition):

```
{  
  "success": false,  
  "message": "Invalid status transition",  
  "errors": {  
    "status": ["Cannot transition from PENDING to DELIVERED"]  
  }  
}
```

6.8 Pickup Slots Endpoints

GET /pickup-slots/

List available pickup slots.

Query Parameters:

- `date` (string, YYYY-MM-DD): Filter by date
- `is_active` (boolean): Filter active slots

Response (200):

```
{  
  "success": true,  
  "message": "Pickup slots retrieved successfully",  
  "data": [  

```

```

    {
      "id": "uuid",
      "date": "2024-08-25",
      "start_time": "14:00",
      "end_time": "16:00",
      "capacity": 20,
      "booked": 15,
      "available": 5,
      "is_active": true
    }
  ]
}

```

Available Capacity Calculation:

```

available = capacity - active_orders
active_orders = orders with status in [PENDING, CONFIRMED, PREPARING, READY_FOR_PICKUP]

```

POST /pickup-slots/ (Admin Only)

Create pickup slot.

PUT /pickup-slots/{id}/ (Admin Only)

Update pickup slot.

DELETE /pickup-slots/{id}/ (Admin Only)

Deactivate pickup slot.

6.9 Returns & Exchanges Endpoints

POST /returns/

Create return or exchange request.

Request:

```

{
  "order_item_id": "uuid",
  "request_type": "RETURN",
  "reason": "Product damaged during delivery",
  "replacement_product_id": null
}

```

Eligibility Validation:

- Order status is DELIVERED or COMPLETED
- Request within 7 days of delivery/completion

- No existing active/completed return for this item
- Valid reason provided

Response (201):

```
{
  "success": true,
  "message": "Return request created successfully",
  "data": {
    "id": "uuid",
    "order_item": {...},
    "request_type": "RETURN",
    "reason": "Product damaged during delivery",
    "status": "REQUESTED",
    "requested_at": "2024-08-21T10:30:00Z"
  }
}
```

GET /returns/

List user's return requests (Customer) or all requests (Staff/Admin).

GET /returns/{id}/

Get return request details.

PATCH /returns/{id}/process/ (Staff/Admin Only)

Process return/exchange request.

Request:

```
{
  "status": "APPROVED",
  "replacement_product_id": "uuid"  // For exchanges only
}
```

Status Options:

- APPROVED
- REJECTED
- COMPLETED (after physical return received)

7. Authentication & Authorization

7.1 JWT Configuration

```
from datetime import timedelta

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=15),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7),
    'ROTATE_REFRESH_TOKENS': True,
    'BLACKLIST_AFTER_ROTATION': True,
    'ALGORITHM': 'HS256',
    'AUTH_HEADER_TYPES': ('Bearer',),
}
```

7.2 Password Requirements

```
AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
        'OPTIONS': {'min_length': 8}
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]
```

Custom Validation:

- Minimum 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one digit
- At least one special character (!@#\$\$%^&*()_+=[{}|;:.,<>?)

7.3 Rate Limiting

```
# Login rate limiting
RATELIMIT_ENABLE = True
RATELIMIT_VIEW = 'apps.accounts.views.login_view'
RATELIMIT_KEY_PREFIX = 'login'
```

```
RATELIMIT_COUNT = 5
RATELIMIT_PERIOD = 900 # 15 minutes
```

7.4 Custom Permissions

```
from rest_framework import permissions

class IsCustomer(permissions.BasePermission):
    def has_permission(self, request, view):
        return request.user.role == 'CUSTOMER'

class IsStaff(permissions.BasePermission):
    def has_permission(self, request, view):
        return request.user.role in ['STAFF', 'ADMIN']

class IsAdmin(permissions.BasePermission):
    def has_permission(self, request, view):
        return request.user.role == 'ADMIN'

class IsOwnerOrReadOnly(permissions.BasePermission):
    def has_object_permission(self, request, view, obj):
        if request.method in ['GET', 'HEAD', 'OPTIONS']:
            return True
        return obj.user == request.user
```

8. Business Logic

8.1 Checkout Transaction Flow

```
from django.db import transaction
from django.core.exceptions import ValidationError
from django.utils import timezone

@transaction.atomic
def process_checkout(user, fulfillment_type, pickup_slot_id=None, delivery_address=None):
    """
    Atomic checkout with proper locking to prevent overselling.
    """
    # 1. Get active cart with items (with row-level locking)
    cart = Cart.objects.select_for_update().get(user=user, is_active=True)
    cart_items = cart.items.select_related('product').all()

    # 2. Validate cart is not empty
    if not cart_items.exists():
        raise ValidationError("Cart is empty")

    # 3. Validate all products and stock
    for item in cart_items:
        if item.quantity > item.product.stock_quantity:
            raise ValidationError(
                f"Insufficient stock for {item.product.name}. "
```

```

        f"Requested: {item.quantity}, Available: {item.product.stock_quantity}
    )

# 4. Validate pickup slot capacity (if applicable)
if fulfillment_type == 'PICKUP':
    slot = PickupSlot.objects.select_for_update().get(id=pickup_slot_id)
    active_orders_count = Order.objects.filter(
        pickup_slot=slot,
        status__in=['PENDING', 'CONFIRMED', 'PREPARING', 'READY_FOR_PICKUP']
    ).count()

    if active_orders_count >= slot.capacity:
        raise ValidationError("This pickup slot is currently full. Please select

# 5. Calculate totals
subtotal = sum(item.quantity * item.product.price for item in cart_items)

# 6. Create order
order = Order.objects.create(
    user=user,
    order_number=generate_order_number(),
    fulfillment_type=fulfillment_type,
    status='PENDING',
    subtotal=subtotal,
    total_amount=subtotal,
    pickup_slot_id=pickup_slot_id if fulfillment_type == 'PICKUP' else None,
    delivery_address=delivery_address if fulfillment_type == 'DELIVERY' else None
)

# 7. Create order items (snapshot product data)
for cart_item in cart_items:
    OrderItem.objects.create(
        order=order,
        product=cart_item.product,
        product_name=cart_item.product.name,
        unit_price=cart_item.product.price,
        quantity=cart_item.quantity,
        subtotal=cart_item.quantity * cart_item.product.price
    )

# 8. Reduce stock
cart_item.product.stock_quantity -= cart_item.quantity
cart_item.product.save(update_fields=['stock_quantity'])

# 9. Clear cart
cart_items.delete()

# 10. Log audit
AuditLog.objects.create(
    user=user,
    action='ORDER_CREATED',
    entity_type='Order',
    entity_id=order.id,
    metadata={
        'order_number': order.order_number,
        'total': str(order.total_amount),
    }
)

```

```

        'items_count': cart_items.count()
    }
)

return order

```

8.2 Order Status Transitions

```

VALID_STATUS_TRANSITIONS = {
    'PENDING': ['CONFIRMED', 'CANCELLED'],
    'CONFIRMED': ['PREPARING', 'CANCELLED'],
    'PREPARING': ['READY_FOR_PICKUP', 'OUT_FOR_DELIVERY'],
    'READY_FOR_PICKUP': ['COMPLETED'],
    'OUT_FOR_DELIVERY': ['DELIVERED'],
    'COMPLETED': [],
    'DELIVERED': [],
    'CANCELLED': [],
}

def validate_status_transition(current_status, new_status):
    """Validate order status transition."""
    allowed = VALID_STATUS_TRANSITIONS.get(current_status, [])
    if new_status not in allowed:
        raise ValidationError(
            f"Invalid status transition from {current_status} to {new_status}"
        )
    return True

```

8.3 Return Eligibility

```

from datetime import timedelta

def validate_return_eligibility(order_item, user):
    """Validate if order item is eligible for return/exchange."""
    # 1. Check order status
    if order_item.order.status not in ['DELIVERED', 'COMPLETED']:
        raise ValidationError("Order must be delivered or completed to request return")

    # 2. Check time window (7 days)
    seven_days_ago = timezone.now() - timedelta(days=7)
    if order_item.order.created_at < seven_days_ago:
        raise ValidationError("Return request must be within 7 days of delivery")

    # 3. Check for existing return requests
    existing_request = ReturnExchangeRequest.objects.filter(
        order_item=order_item,
        status__in=['REQUESTED', 'APPROVED', 'COMPLETED']
    ).exists()

    if existing_request:
        raise ValidationError("This item already has an active or completed return request")

    # 4. Check user ownership

```

```
if order_item.order.user != user:
    raise ValidationError("You can only request returns for your own orders")

return True
```

8.4 Order Number Generation

```
from datetime import datetime

def generate_order_number():
    """Generate unique order number: ORD-YYYY-NNNNNN"""
    date_str = datetime.now().strftime('%Y')
    count = Order.objects.filter(order_number__startswith=f'ORD-{date_str}-').count()
    sequence = str(count + 1).zfill(6)
    return f'ORD-{date_str}-{sequence}'
```

9. Security Requirements

9.1 Authentication Security

- **Password Hashing:** Django PBKDF2 with 600,000 iterations
- **JWT Tokens:**
 - Access token: 15 minutes
 - Refresh token: 7 days
 - Token rotation on refresh
- **Session Security:** Redis-backed sessions with secure cookies

9.2 Authorization

- **RBAC:** Custom DRF permissions for each role
- **Resource Ownership:** All user-specific resources validated against `request.user`
- **IDOR Prevention:** UUIDs for public-facing IDs, integer IDs internally
- **Vertical Privilege Escalation:** Role checks on every sensitive endpoint

9.3 Input Validation

- **Serializer Validation:** All DRF serializers with field validators
- **SQL Injection:** Django ORM parameterized queries (no raw SQL)
- **XSS Prevention:** React's built-in escaping, Django template auto-escaping
- **CSRF Protection:** Django CSRF middleware for session-based auth

9.4 Security Headers

```
SECURE_BROWSER_XSS_FILTER = True
SECURE_CONTENT_TYPE_NOSNIFF = True
X_FRAME_OPTIONS = 'DENY'
SECURE_REFERRER_POLICY = 'strict-origin-when-cross-origin'
SECURE_SSL_REDIRECT = True # Production only
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
```

9.5 Audit Logging

Events to Log:

- User registration, login, logout
- Password changes
- Role changes
- Product CRUD operations
- Inventory adjustments
- Order status changes
- Return/exchange approvals/rejections
- Admin dashboard access
- Failed authentication attempts

Log Structure:

```
{
  "user_id": "uuid",
  "action": "ORDER_STATUS_CHANGED",
  "entity_type": "Order",
  "entity_id": "uuid",
  "metadata": {
    "old_status": "PENDING",
    "new_status": "CONFIRMED",
    "ip_address": "192.168.1.1"
  },
  "timestamp": "2024-08-21T10:30:00Z"
}
```

10. Testing Strategy

10.1 Coverage Requirements

Test Type	Minimum Coverage
Overall	80%
Critical Paths	95%+
Backend	85%
Frontend	75%

10.2 Test Categories

Unit Tests:

- Model methods and properties
- Serializer validation
- Service layer business logic
- Utility functions

Integration Tests:

- API endpoint responses
- Database transactions
- Authentication flows
- Permission checks

End-to-End Tests:

- Complete user registration → checkout flow
- Staff order processing workflow
- Admin product management
- Return/exchange request flow

Security Tests:

- SQL injection attempts
- XSS payload injection
- JWT token tampering
- Privilege escalation attempts
- Rate limiting effectiveness
- IDOR vulnerability scanning

Performance Tests:

- Concurrent checkout (100 simultaneous users)
- Product listing with filters (10,000 products)

- Order history pagination

10.3 Example Test Cases

Authentication Tests:

```
def test_valid_login():
    """Test successful login with valid credentials."""

def test_invalid_login():
    """Test login with invalid credentials."""

def test_login_rate_limiting():
    """Test rate limiting after 5 failed attempts."""

def test_expired_jwt_token():
    """Test API access with expired token."""

def test_role_based_access():
    """Test customer cannot access admin endpoints."""
```

Cart Tests:

```
def test_add_to_cart():
    """Test adding product to cart."""

def test_add_exceeds_stock():
    """Test adding quantity exceeding stock."""

def test_update_cart_quantity():
    """Test updating cart item quantity."""

def test_clear_cart():
    """Test clearing entire cart."""
```

Checkout Tests:

```
def test_successful_checkout():
    """Test complete checkout flow."""

def test_checkout_empty_cart():
    """Test checkout with empty cart."""

def test_checkout_insufficient_stock():
    """Test checkout when stock is insufficient."""

def test_checkout_full_pickup_slot():
    """Test checkout when pickup slot is full."""

def test_concurrent_checkout():
    """Test 100 simultaneous checkouts."""
```

Order Tests:

```
def test_valid_status_transition():
    """Test valid order status transitions."""

def test_invalid_status_transition():
    """Test invalid status transitions."""

def test_order_cancellation():
    """Test customer order cancellation."""

def test_unauthorized_order_access():
    """Test customer accessing another customer's order."""
```

Return Tests:

```
def test_eligible_return():
    """Test return within 7 days."""

def test_expired_return():
    """Test return after 7 days."""

def test_duplicate_return():
    """Test duplicate return request."""
```

11. Deployment Configuration

11.1 Docker Compose (Development)

```
version: '3.8'

services:
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: mini_dmart
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: dev_password
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    ports:
```

```

    - "6379:6379"
healthcheck:
  test: ["CMD", "redis-cli", "ping"]
  interval: 10s
  timeout: 5s
  retries: 5

backend:
  build:
    context: ./backend
    dockerfile: ../docker/Dockerfile.backend
  command: python manage.py runserver 0.0.0.0:8000
  volumes:
    - ./backend:/app
  ports:
    - "8000:8000"
  environment:
    - DATABASE_URL=postgres://postgres:dev_password@db:5432/mini_dmart
    - REDIS_URL=redis://redis:6379/0
    - DEBUG=True
    - SECRET_KEY=dev_secret_key
  depends_on:
    db:
      condition: service_healthy
    redis:
      condition: service_healthy

frontend:
  build:
    context: ./frontend
    dockerfile: ../docker/Dockerfile.frontend
  command: npm run dev -- --host
  volumes:
    - ./frontend:/app
    - /app/node_modules
  ports:
    - "5173:5173"
  environment:
    - VITE_API_URL=http://localhost:8000/api/v1
  depends_on:
    - backend

volumes:
  postgres_data:

```

11.2 GitHub Actions CI/CD

```

name: CI/CD Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

```

```

jobs:
  test-backend:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_PASSWORD: postgres
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
      redis:
        image: redis:7-alpine
        options: >-
          --health-cmd "redis-cli ping"
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
      - uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          cd backend
          pip install -r requirements/development.txt

      - name: Run migrations
        run: |
          cd backend
          python manage.py migrate

      - name: Run tests
        run: |
          cd backend
          pytest --cov=apps --cov-report=xml --cov-report=html

      - name: Upload coverage
        uses: codecov/codecov-action@v3
        with:
          files: ./backend/coverage.xml
          flags: backend

  test-frontend:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

```

```

- name: Set up Node.js
  uses: actions/setup-node@v3
  with:
    node-version: '20'

- name: Install dependencies
  run: |
    cd frontend
    npm ci

- name: Run linting
  run: |
    cd frontend
    npm run lint

- name: Run tests
  run: |
    cd frontend
    npm run test:coverage

- name: Build
  run: |
    cd frontend
    npm run build

deploy:
  needs: [test-backend, test-frontend]
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'

  steps:
    - uses: actions/checkout@v3

    - name: Deploy to Render
      uses: johnbeynon/render-deploy-action@v0.0.8
      with:
        service-id: ${ secrets.RENDER_SERVICE_ID }
        api-key: ${ secrets.RENDER_API_KEY }

    - name: Deploy to Vercel
      uses: amondnet/vercel-action@v20
      with:
        vercel-token: ${ secrets.VERCEL_TOKEN }
        vercel-org-id: ${ secrets.VERCEL_ORG_ID }
        vercel-project-id: ${ secrets.VERCEL_PROJECT_ID }
        working-directory: ./frontend

```

11.3 Environment Variables

Backend .env.example:

```

# Django
SECRET_KEY=your-secret-key-here
DEBUG=True
ALLOWED_HOSTS=localhost,127.0.0.1

```

```
# Database
DATABASE_URL=postgres://postgres:password@localhost:5432/mini_dmart
DATABASE_NAME=mini_dmart
DATABASE_USER=postgres
DATABASE_PASSWORD=your_password
DATABASE_HOST=localhost
DATABASE_PORT=5432

# Redis
REDIS_URL=redis://localhost:6379/0

# JWT
JWT_SECRET_KEY=your-jwt-secret
JWT_ACCESS_TOKEN_LIFETIME=15
JWT_REFRESH_TOKEN_LIFETIME=7

# CORS
CORS_ALLOWED_ORIGINS=http://localhost:5173,http://127.0.0.1:5173

# Security
SECURE_SSL_REDIRECT=False
SESSION_COOKIE_SECURE=False
CSRF_COOKIE_SECURE=False

# Email (for password reset)
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_HOST_USER=your-email@gmail.com
EMAIL_HOST_PASSWORD=your-app-password
EMAIL_USE_TLS=True

# Sentry (optional)
SENTRY_DSN=your-sentry-dsn
```

Frontend .env.example:

```
VITE_API_URL=http://localhost:8000/api/v1
VITE_APP_NAME=Mini D-Mart
```

12. Performance Requirements

12.1 SLA Targets

Metric	Target
API Response Time (95th percentile)	< 200ms
Page Load Time (3G)	< 3 seconds
Time to First Byte (TTFB)	< 500ms
Uptime	99.9%

Metric	Target
Concurrent Checkouts	100+ users

12.2 Caching Strategy

Data Type	Cache Duration	Backend
Product Listing	5 minutes	Redis
Category List	1 hour	Redis
User Sessions	7 days	Redis
Rate Limiting	15 minutes	Redis
Pickup Slots	2 minutes	Redis

12.3 Database Optimization

- **Connection Pooling:** 20 connections (dev), 100+ (production)
- **Query Optimization:** `select_related` and `prefetch_related` for N+1 prevention
- **Indexing:** All foreign keys, frequently filtered fields
- **Query Logging:** Enable in development, sample in production

12.4 Frontend Optimization

- **Code Splitting:** Route-based lazy loading
- **Image Optimization:** WebP format, lazy loading
- **Bundle Size:** < 500KB initial load
- **Tree Shaking:** Remove unused code
- **CDN:** Static assets via Vercel CDN

13. Monitoring & Observability

13.1 Health Check Endpoints

```
GET /api/v1/health/

Response (200):
{
  "status": "healthy",
  "timestamp": "2024-08-21T10:30:00Z",
  "services": {
    "database": "ok",
    "redis": "ok",
    "cache": "ok"
  },
}
```

```
"version": "1.0.0"
}
```

13.2 Logging Configuration

```
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'json': {
            '()': 'pythonjsonlogger.jsonlogger.JsonFormatter'
        }
    },
    'handlers': {
        'console': {
            'class': 'logging.StreamHandler',
            'formatter': 'json'
        },
        'file': {
            'class': 'logging.handlers.RotatingFileHandler',
            'filename': 'logs/app.log',
            'maxBytes': 10485760, # 10MB
            'backupCount': 5,
            'formatter': 'json'
        }
    },
    'root': {
        'handlers': ['console', 'file'],
        'level': 'INFO'
    },
    'loggers': {
        'django': {
            'handlers': ['console', 'file'],
            'level': 'INFO',
            'propagate': False,
        },
    }
}
```

13.3 Error Tracking

- **Sentry Integration:** All unhandled exceptions captured
- **Correlation IDs:** Included in all logs for request tracing
- **Alerting:** Slack/email notifications for critical errors

13.4 Metrics to Track

- API response times (p50, p95, p99)
- Error rates by endpoint
- Database query times
- Cache hit/miss ratios
- Active users
- Orders per hour
- Checkout success rate
- Cart abandonment rate

14. Development Phases

Phase 1: Project Setup (Days 1-2)

Tasks:

- ☐ Initialize Git repository with proper .gitignore
- ☐ Set up Django project with separated settings
- ☐ Configure PostgreSQL with Docker
- ☐ Set up React + Vite + TypeScript
- ☐ Configure Tailwind CSS
- ☐ Set up ESLint, Prettier, Black, isort
- ☐ Create Docker Compose for local development
- ☐ Set up GitHub Actions CI pipeline
- ☐ Create README.md with setup instructions

Deliverables:

- Working development environment
- CI pipeline running tests
- Basic project structure

Phase 2: Authentication (Days 3-5)

Tasks:

- ☐ Custom User model with roles
- ☐ JWT authentication setup
- ☐ Registration with validation
- ☐ Login with rate limiting

- ☐ Token refresh mechanism
- ☐ Profile management
- ☐ RBAC permissions
- ☐ Protected routes (frontend + backend)
- ☐ Write authentication tests

Deliverables:

- Working registration/login
- JWT token management
- Role-based access control
- Test coverage > 85%

Phase 3: Products & Categories (Days 6-8)

Tasks:

- ☐ Category model and CRUD
- ☐ Product model with validations
- ☐ Product listing with pagination
- ☐ Search functionality
- ☐ Category and price filters
- ☐ Product detail endpoint
- ☐ Admin product management
- ☐ Image upload handling
- ☐ Low-stock threshold alerts
- ☐ Write product tests

Deliverables:

- Product catalog API
- Search and filter functionality
- Admin product management UI
- Test coverage > 80%

Phase 4: Shopping Cart (Days 9-10)

Tasks:

- ☐ Cart model (one per user)
- ☐ CartItem model with unique constraint
- ☐ Add to cart API

- ☐ Update quantity API
- ☐ Remove from cart API
- ☐ Clear cart API
- ☐ Cart calculations (subtotal, total items)
- ☐ Stock validation on add/update
- ☐ Write cart tests

Deliverables:

- Full cart management API
- Real-time stock validation
- Cart UI components
- Test coverage > 85%

Phase 5: Checkout & Orders (Days 11-14)

Tasks:

- ☐ Order model with snapshot
- ☐ OrderItem model
- ☐ PickupSlot model
- ☐ Checkout endpoint with transactions
- ☐ Stock reservation
- ☐ Order creation
- ☐ Order history endpoint
- ☐ Order detail endpoint
- ☐ Order cancellation
- ☐ Write checkout tests

Deliverables:

- Complete checkout flow
- Order management
- Pickup slot system
- Test coverage > 90%

Phase 6: Order Operations (Days 15-17)

Tasks:

- ☐ Staff order dashboard
- ☐ Status transition logic

- ☐ Order status update API
- ☐ Pickup order management
- ☐ Delivery order management
- ☐ Customer order tracking
- ☐ Write operations tests

Deliverables:

- Staff order processing UI
- Status workflow
- Order tracking
- Test coverage > 85%

Phase 7: Returns & Exchanges (Days 18-20)

Tasks:

- ☐ ReturnExchangeRequest model
- ☐ Return request API
- ☐ Exchange request API
- ☐ Eligibility validation
- ☐ Approval/rejection workflow
- ☐ Inventory restoration on approval
- ☐ Write returns tests

Deliverables:

- Return/exchange request system
- Approval workflow
- Test coverage > 85%

Phase 8: Dashboards (Days 21-23)

Tasks:

- ☐ Customer dashboard with stats
- ☐ Staff dashboard with operational metrics
- ☐ Admin dashboard with business metrics
- ☐ Inventory alerts
- ☐ Low-stock product list
- ☐ Audit log viewer
- ☐ Write dashboard tests

Deliverables:

- Three role-specific dashboards
- Analytics and metrics
- Audit logging UI
- Test coverage > 80%

Phase 9: Security & Testing (Days 24-26)**Tasks:**

- ☐ Security audit
- ☐ Penetration testing
- ☐ Fix identified vulnerabilities
- ☐ Complete test coverage
- ☐ Performance testing
- ☐ Load testing
- ☐ Write SECURITY.md

Deliverables:

- Security audit report
- All tests passing
- Performance benchmarks
- SECURITY.md documentation

Phase 10: Deployment (Days 27-28)**Tasks:**

- ☐ Production database setup
- ☐ Environment configuration
- ☐ Backend deployment
- ☐ Frontend deployment
- ☐ Production testing
- ☐ Monitoring setup
- ☐ Documentation finalization

Deliverables:

- Live production environment
- Monitoring dashboards
- Complete documentation

15. Frontend Routes

15.1 Public Routes

Route	Component	Description
/	HomePage	Landing page with featured products
/login	LoginPage	User login
/register	RegisterPage	User registration
/products	ProductsPage	Product catalog with filters
/products/:id	ProductDetailPage	Product details

15.2 Customer Routes (Protected)

Route	Component	Description
/cart	CartPage	Shopping cart
/checkout	CheckoutPage	Checkout flow
/orders	OrdersPage	Order history
/orders/:id	OrderDetailPage	Order details
/returns	ReturnsPage	Return/exchange requests
/profile	ProfilePage	User profile

15.3 Staff Routes (Protected)

Route	Component	Description
/staff	StaffDashboard	Staff dashboard
/staff/orders	StaffOrdersPage	All orders
/staff/pickups	StaffPickupsPage	Upcoming pickups
/staff/deliveries	StaffDeliveriesPage	Delivery orders
/staff/returns	StaffReturnsPage	Return requests

15.4 Admin Routes (Protected)

Route	Component	Description
/admin	AdminDashboard	Admin dashboard
/admin/products	AdminProductsPage	Product management
/admin/categories	AdminCategoriesPage	Category management

Route	Component	Description
/admin/users	AdminUsersPage	User management
/admin/orders	AdminOrdersPage	All orders
/admin/inventory	AdminInventoryPage	Inventory management
/admin/audit-logs	AdminAuditLogsPage	Audit logs
/admin/pickup-slots	AdminPickupSlotsPage	Pickup slot management

15.5 Route Protection

```
// ProtectedRoute.tsx
interface ProtectedRouteProps {
  children: React.ReactNode;
  requiredRole?: 'CUSTOMER' | 'STAFF' | 'ADMIN';
}

const ProtectedRoute: React.FC<ProtectedRouteProps> = ({
  children,
  requiredRole
}) => {
  const { user, isAuthenticated } = useAuth();

  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }

  if (requiredRole && user.role !== requiredRole) {
    return <Navigate to="/" replace />;
  }

  return <>{children}</>;
};
```

16. UI/UX Requirements

16.1 Design Principles

- **Modern:** Clean, contemporary design
- **Professional:** Business-appropriate aesthetics
- **Responsive:** Mobile-first, works on all devices
- **Accessible:** WCAG 2.1 AA compliance
- **Consistent:** Unified design system

16.2 Required Components

Layout:

- Navbar (responsive, role-aware)
- Sidebar (staff/admin)
- Footer
- ProtectedRoute wrapper
- RoleProtectedRoute wrapper

UI Primitives:

- Button (primary, secondary, danger)
- Input (text, email, password, number)
- Select/Dropdown
- Modal/Dialog
- Toast notifications
- Loader/Spinner
- Badge/Status indicator
- Card
- Table
- Pagination

Feature Components:

- ProductCard
- ProductGrid
- ProductFilters
- ProductSearch
- SearchBar
- CartItem
- CartSummary
- EmptyCart
- EmptyState
- OrderCard
- OrderStatus
- OrderTimeline
- ReturnRequestForm
- ReturnStatus
- StatusBadge

- FilterPanel
- ErrorMessage
- SuccessMessage
- ConfirmationDialog

16.3 Loading States

- Skeleton loaders for product cards
- Spinner for async actions
- Progress bar for multi-step processes
- Inline loading for infinite scroll

16.4 Empty States

- Empty cart illustration
- No orders message
- No products found
- No search results
- No return requests

16.5 Error States

- Form validation errors (inline)
- API error messages (toast)
- Network error handling
- 404 page
- 500 error page

16.6 Success Notifications

- Order placed confirmation
- Item added to cart
- Profile updated
- Return request submitted
- Order cancelled

16.7 Confirmation Dialogs

- Cancel order confirmation
- Remove from cart confirmation
- Clear cart confirmation
- Logout confirmation
- Delete product confirmation (admin)

17. Known Limitations (MVP Scope)

The following features are intentionally excluded from MVP but documented for future iterations:

1. **Payment Gateway Integration:** Orders are created without actual payment processing
2. **Email Notifications:** No automated emails for order confirmations, status updates
3. **SMS Notifications:** No SMS alerts for pickup readiness
4. **Product Reviews:** Customer reviews and ratings not implemented
5. **Wishlist:** Save-for-later functionality not included
6. **Promotions/Discounts:** No coupon or discount system
7. **Multi-address Delivery:** Single delivery address per order
8. **Scheduled Orders:** No recurring or scheduled orders
9. **Gift Wrapping:** No gift options
10. **Loyalty Program:** No points or rewards system
11. **Advanced Search:** No autocomplete or suggestions
12. **Product Recommendations:** No ML-based recommendations
13. **Mobile Apps:** Web-only, no native iOS/Android apps
14. **Multi-language:** English only
15. **Multi-currency:** Single currency (USD)
16. **Advanced Analytics:** Basic dashboards only
17. **Inventory Forecasting:** No predictive analytics
18. **Multi-store Support:** Single store location
19. **Customer Support Chat:** No live chat
20. **Social Login:** Email/password only

18. Getting Started

18.1 Prerequisites

- Python 3.11+
- Node.js 20+
- Docker & Docker Compose
- PostgreSQL 15+ (or use Docker)
- Redis 7+ (or use Docker)
- Git

18.2 Quick Start

```
# Clone repository
git clone https://github.com/your-org/mini-dmart.git
cd mini-dmart

# Start all services (database, redis, backend, frontend)
docker-compose up -d

# Backend setup (if not using Docker)
cd backend
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements/development.txt
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver

# Frontend setup (if not using Docker)
cd frontend
npm install
npm run dev

# Access application
# Frontend: http://localhost:5173
# Backend API: http://localhost:8000/api/v1/
# API Docs: http://localhost:8000/api/v1/schema/
```

18.3 Database Setup

```
# Create database
createdb mini_dmart

# Run migrations
python manage.py migrate

# Create superuser
python manage.py createsuperuser
```

```
# Load sample data (optional)
python manage.py loaddata sample_data.json
```

18.4 Running Tests

```
# Backend tests
cd backend
pytest --cov=apps

# Frontend tests
cd frontend
npm run test

# E2E tests
cd frontend
npm run test:e2e
```

18.5 Development Commands

Backend:

```
# Run server
python manage.py runserver

# Run migrations
python manage.py migrate

# Create migrations
python manage.py makemigrations

# Collect static files
python manage.py collectstatic

# Run tests
pytest

# Format code
black apps/
isort apps/

# Type checking
mypy apps/
```

Frontend:

```
# Development server
npm run dev

# Build for production
npm run build
```

```
# Run tests
npm run test

# Run linting
npm run lint

# Format code
npm run format
```

19. Test Credentials

19.1 Pre-configured Test Accounts

Customer Account:

```
Email: customer@test.com
Password: Test@123456
Role: CUSTOMER
```

Staff Account:

```
Email: staff@test.com
Password: Test@123456
Role: STAFF
```

Admin Account:

```
Email: admin@test.com
Password: Test@123456
Role: ADMIN
```

19.2 Sample Data

The application includes sample data for testing:

- 5 categories
- 50 products
- 10 pickup slots
- Sample orders for each status

20. Success Criteria

20.1 Functional Completeness

- ✓ All user stories implemented and tested
- ✓ All API endpoints functional with proper error handling
- ✓ All business rules enforced at backend level
- ✓ No critical or high-severity bugs
- ✓ All acceptance criteria met

20.2 Security

- ✓ OWASP Top 10 vulnerabilities addressed
- ✓ All authentication and authorization working correctly
- ✓ No sensitive data exposure
- ✓ Rate limiting effective
- ✓ Input validation comprehensive
- ✓ Audit logging functional

20.3 Performance

- ✓ All pages load in < 3 seconds
- ✓ API responses under 200ms (95th percentile)
- ✓ Concurrent checkout handles 100 users without overselling
- ✓ Database queries optimized
- ✓ Caching implemented correctly

20.4 Code Quality

- ✓ 80%+ test coverage
- ✓ No linting errors
- ✓ Clean, documented code
- ✓ Proper error handling throughout
- ✓ Type safety (TypeScript, mypy)
- ✓ Consistent code style

20.5 Documentation

- ✓ README with setup instructions
- ✓ API documentation (OpenAPI/Swagger)
- ✓ SECURITY.md with audit findings
- ✓ Deployment guide
- ✓ Architecture documentation
- ✓ CHANGELOG.md

20.6 User Experience

- ✓ Responsive design (mobile, tablet, desktop)
- ✓ Intuitive navigation
- ✓ Clear error messages
- ✓ Loading states
- ✓ Empty states
- ✓ Success notifications
- ✓ Accessible (WCAG 2.1 AA)

21. Final Deliverables

21.1 Code & Configuration

1. Complete Source Code:

- Frontend (React + TypeScript)
- Backend (Django + DRF)
- Database migrations
- Docker configuration
- CI/CD pipelines

2. Environment Files:

- `.env.example` (backend)
- `.env.example` (frontend)
- `docker-compose.yml`

3. Dependencies:

- `requirements/base.txt`
- `requirements/development.txt`
- `requirements/production.txt`
- `package.json`

21.2 Documentation

4. README.md:

- Project overview
- Features list
- Technology stack
- Installation instructions
- Development setup
- Testing instructions
- Deployment guide

5. SECURITY.md:

- Authentication security
- RBAC implementation
- Input validation
- Rate limiting
- Security testing performed
- Security findings
- Issues found and fixed
- Remaining security limitations

6. CHANGELOG.md:

- Version history
- Feature additions
- Bug fixes
- Breaking changes

7. CONTRIBUTING.md:

- Code style guide
- Git workflow
- Pull request process
- Issue reporting

8. **API Documentation:**

- OpenAPI/Swagger schema
- Interactive API docs
- Example requests/responses

21.3 Testing

9. Test Suite:

- Unit tests
- Integration tests
- E2E tests
- Security tests
- Performance tests

10. Test Coverage Report:

- HTML coverage report
- Coverage badges

21.4 Deployment

11. Deployment Configuration:

- Docker files
- Docker Compose
- GitHub Actions workflows
- Environment-specific settings

12. Monitoring Setup:

- Health check endpoints
- Logging configuration
- Error tracking (Sentry)

21.5 Sample Data

13. Test Data:

- Sample categories
- Sample products
- Sample pickup slots
- Test user accounts

Appendix A: API Quick Reference

Authentication

- POST /api/v1/auth/register/ - Register new user
- POST /api/v1/auth/login/ - Login
- POST /api/v1/auth/token/refresh/ - Refresh token
- GET /api/v1/auth/profile/ - Get profile
- PUT /api/v1/auth/profile/ - Update profile

Products

- GET /api/v1/categories/ - List categories
- GET /api/v1/products/ - List products
- GET /api/v1/products/{id}/ - Get product
- POST /api/v1/products/ - Create product (Admin)
- PUT /api/v1/products/{id}/ - Update product (Admin)
- DELETE /api/v1/products/{id}/ - Delete product (Admin)

Cart

- GET /api/v1/cart/ - Get cart
- POST /api/v1/cart/items/ - Add to cart
- PATCH /api/v1/cart/items/{id}/ - Update cart item
- DELETE /api/v1/cart/items/{id}/ - Remove from cart
- DELETE /api/v1/cart/clear/ - Clear cart

Orders

- POST /api/v1/orders/checkout/ - Checkout
- GET /api/v1/orders/ - List orders
- GET /api/v1/orders/{id}/ - Get order
- POST /api/v1/orders/{id}/cancel/ - Cancel order
- PATCH /api/v1/orders/{id}/status/ - Update status (Staff/Admin)

Pickup Slots

- GET /api/v1/pickup-slots/ - List slots
- POST /api/v1/pickup-slots/ - Create slot (Admin)
- PUT /api/v1/pickup-slots/{id}/ - Update slot (Admin)
- DELETE /api/v1/pickup-slots/{id}/ - Delete slot (Admin)

Returns

- POST /api/v1/returns/ - Create return request
- GET /api/v1/returns/ - List returns
- GET /api/v1/returns/{id}/ - Get return
- PATCH /api/v1/returns/{id}/process/ - Process return (Staff/Admin)

Admin

- GET /api/v1/admin/dashboard/ - Admin dashboard
- GET /api/v1/admin/users/ - List users
- PATCH /api/v1/admin/users/{id}/role/ - Update role
- GET /api/v1/admin/audit-logs/ - Audit logs

Appendix B: Troubleshooting

Common Issues

Database Connection Error:

```
# Check PostgreSQL is running
docker ps | grep postgres

# Verify connection string
echo $DATABASE_URL

# Test connection
psql $DATABASE_URL
```

CORS Error:

```
# Add frontend origin to CORS_ALLOWED_ORIGINS
CORS_ALLOWED_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
```

JWT Token Expired:

```
// Implement token refresh in axios interceptor
axios.interceptors.response.use(
  response => response,
  async error => {
    if (error.response?.status === 401) {
      // Refresh token and retry
    }
  }
);
```

Migration Errors:

```
# Reset migrations (development only)
python manage.py migrate app_name zero
python manage.py migrate
```

Port Already in Use:

```
# Find process using port
lsof -i :8000

# Kill process
kill -9 <PID>
```

Appendix C: Glossary

Term	Definition
RBAC	Role-Based Access Control
JWT	JSON Web Token
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
IDOR	Insecure Direct Object Reference
XSS	Cross-Site Scripting
CSRF	Cross-Site Request Forgery
SLA	Service Level Agreement
TTFB	Time to First Byte
E2E	End-to-End
CI/CD	Continuous Integration/Continuous Deployment
ERD	Entity Relationship Diagram
UUID	Universally Unique Identifier
ORM	Object-Relational Mapping
N+1 Query	Database anti-pattern

Appendix D: Changelog Template

[1.0.0] - 2024-08-21

Added:

- Initial release
- User authentication and authorization
- Product catalog with search and filters
- Shopping cart management
- Checkout system with pickup and delivery
- Order management
- Returns and exchanges
- Role-based dashboards
- Audit logging

Changed:

- N/A

Fixed:

- N/A

Security:

- Implemented JWT authentication
- Added rate limiting
- Comprehensive input validation
- RBAC enforcement

Appendix E: Security Checklist

- ☐ Password hashing (PBKDF2)
- ☐ JWT token expiration
- ☐ Rate limiting on auth endpoints
- ☐ Input validation on all endpoints
- ☐ SQL injection prevention (ORM only)
- ☐ XSS prevention (React escaping)
- ☐ CSRF protection
- ☐ CORS configuration
- ☐ Security headers

- ☐ Audit logging
- ☐ IDOR prevention
- ☐ Privilege escalation prevention
- ☐ Sensitive data encryption
- ☐ Secure cookie settings
- ☐ Error handling (no stack traces)
- ☐ Logging (no passwords/tokens)

Document End

This specification document is comprehensive and production-ready. It provides all necessary details for building a complete, secure, and scalable Mini D-Mart grocery e-commerce platform.

Last Updated: August 21, 2026

Version: 1.0.0

Status: Production-Ready